

Using Objects

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Poll: Why does this test fail?

```
class Rectangle:

    def __init__(self, length: int, width: int) -> None:
        self.length = length
        self.width = width

class TestRectangle:
    def test_length_width(self) -> None:
        rect1 = Rectangle(3, 4)
        rect2 = Rectangle(4, 3)
        assert rect1 == rect2
```

1. Because you can't put tests inside a class like that.
2. Because the length and width are switched in rect1 and rect2. If they were both (3, 4), then it would pass.
3. Because `==` calls the `eq()` method, and `Rectangle` doesn't have an

Before overwriting `__eq__()` :

```
class Student():
    def __init__(self,
                  student_id: str, major: str
    ):
        self.id = student_id
        self.major = major
        self.courses: set[str] = set()

s1 = Student('s1', 'CS')
s2 = Student('s1', 'CS')
print(s1 == s2)  # False
```

After overwriting `__eq__()` :

```
class Student():
    def __init__(self,
                  student_id: str, major: str
    ):
        self.id = student_id
        self._major = major

    def __eq__(
        self, other: object
    ) -> bool:
        if not isinstance(other, Student):
            return False
        return self.id == other.id

s1 = Student('s1', 'CS')
s2 = Student('s1', 'CS')
print(s1 == s2)  # True
```

Poll: Why is this bad?

```
class Cat:
    def __init__(self, name: str):
        self.name = name
        self.food: list[str] = ['tuna', 'chicken']

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Cat):
            return False
        return other.name in self.food
```

1. It's possible for `cat_a` to equal `cat_b` today, but for `cat_a` to not be equal to `cat_b` tomorrow (with no code changes)
2. It's possible for `cat_a` to not equal itself
3. It's possible for `cat_a` to equal `cat_b`, and `cat_b` to not equal `cat_a`
4. All cats will be equal, making the `__eq__()` function useless

The `__str__()` function

Every class has a `__str__()` method.

We can overwrite it with our own `__str__()` method.

When we print an object, it implicitly calls the `__str__()` method.

The default `__str__()` method is not helpful.

```
mini = Cat('Mini', 'Rasika')  
print(mini) # <__main__.Cat object at 0x1095ca790
```

The `__str__()` function

Why is it different when we print a list (which is also an object)?

We usually write our own `__str__()` method that returns a more helpful `str` for that class.

```
class Cat:
    ...
    def __str__(self):
        return f'{self.name} meows to {self.human}'

mini = Cat('Mini', 'Rasika')
print(mini) # Mini meows to Rasika
```

Poll: What is printed?

```
class Cat:
    def __init__(self, name: str, human: str):
        self.name = name
        self.human = human

    def __str__(self) -> str:
        print('MUAHAHAHAHHA')
        return self.name

mini: Cat = Cat('Mini', 'Rasika')
print(mini)
```

1. Mini
2. MUAHAHAHAHHA // Mini
3. <__main__.Cat object at 0x10d380200>
4. MUAHAHAHAHHA // <__main__.Cat object at 0x10d380200>

The Single Responsibility Principle

A core principle for writing code is that every component of our code must have a single purpose. This makes our code easier to read, test, and maintain.

Component	Does not Follow Single Responsibility Principle
Variable	<code>age_or_nonexistent: int = -1 # negative if nonexistent</code>
Function	<code>def read_file_compute_average_print_score(filename: str) -> None:</code>
Class	<code>class FileManagerAndOutputFormatter</code>

Abstraction

We organize lines of code into functions, functions into classes, classes into...

"Abstraction": the procedure of grouping more granular things into less granular groups

Benefits of abstraction we already saw in functions:

- reuse code without redundancy
- hide implementation details
- break down a problem into smaller, more manageable pieces

The same applies to the idea of putting methods into classes.

Abstraction

Hard to debug or modify:

```
length: int = 5
width: int = 3

print(get_area_of_rectangle(
    length, width))

length = 6
width = 4

print(get_perimeter_of_rectangle(
    length, width))
```

Put relevant perimeter and area methods into a class for each shape:

```
table: Rectangle = Rectangle(5, 3)
print(table.area())

print(Rectangle(6, 4).perimeter())

chair: Square = Square(4)
print(chair.area())
```

Benefits of abstraction:

- We can use the same code multiple times without re-writing it
- It's easier to read
- It's easier to maintain and adapt the code later on
- It's easier to test behaviors in isolation
- Each variable, function, and class has a single, clear responsibility

Generic types

In Python:

We are able to put objects of different types into the same `list`, but it's discouraged (makes the list harder to process).

In CS2100:

Elements of a list must be of the same type since we require types in our Python code.

What would be the type of the variable `my_list = [1, 'a']` ?

The same `list` class can make objects of different types (`list[str]` and `list[int]`) because `list` is a *generic* type.

Define our own generic type:

1. First define the type variable `T`
2. Then use `T` to define the generic type `Stack[T]`
3. Inside the class `Stack[T]`, the `T` can be any type, but all instances of `T` must be the same type as each other
4. Instantiate it as `my_stack`, with `T` taking the value `int`
5. Instantiate another variable `my_other_stack`, where `T` is `str`

```
from typing import TypeVar, Generic

T = TypeVar('T')

class Stack(Generic[T]):
    def __init__(self) -> None:
        self.items: list[T] = []

    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()

    def is_empty(self) -> bool:
        return not self.items

my_stack: Stack[int] = Stack()
my_stack.push(4)
print(my_stack.pop())    # 4
my_other_stack: Stack[str] = Stack()
```

Definitions

- **Generic type:** a class with a type variable, like `list[T]`
- **Parameterized type:** a generic type with the type variables filled in, like `list[str]`
- **Raw type:** a generic type without the type variable, like `list`
 - use this if we don't need to re-use the type variable anywhere else in the code

We can parametrize the type using another user-defined type:

```
stack_of_stacks: Stack[Stack[int]] = Stack()
```

Poll: Which of these is allowed?

```
class Thing(Generic[T]):  
    def __init__(self, item: Optional[T]):  
        """Item is of type T or None"""  
        self.item = item
```

1. `item: Thing[str] = Thing('hello')`
2. `item: Thing[str] = Thing(None)`
3. `item: Thing[str] = Thing(5)`
4. `item: Thing[Thing[str]] = Thing(Thing('hello'))`

Functions with lots of arguments:

```
def display_text(  
    text: str, size: int, is_bold: bool,  
    is_italic: bool, is_underlined: bool) -> None:  
    ...  
  
display_text('hello', 18, False, False, False)  
display_text('goodbye', 18, True, False, False)
```

- **Pros:** multiple options in the same function without compromising flexibility
- **Cons:** error prone, must keep track of order of arguments, too many things to specify each time we call the function

Two solutions: named args and default arg values

Named arguments

```
def display_text(  
    text: str, size: int, is_bold: bool,  
    is_italic: bool, is_underlined: bool) -> None:  
    ...  
  
display_text(  
    text = 'hello', is_underlined = False,  
    is_bold = False, is_italic = False, size = 18)
```

- Make calls more readable
- Enable you to reorder arguments

Default argument values

```
def display_text(  
    text: str, size: int = 18, is_bold: bool = False,  
    is_italic: bool = False, is_underlined: bool = False  
) -> None:  
    ...  
  
display_text(text = 'hello', is_bold = True)
```

- If you usually pass the same value
- Default value when the client doesn't specify a value when calling it
- Args with default values must come after args without default values
- If we have a function that is already widely used, and we want to add another parameter, give it a default value (so the existing code doesn't break)

Default arg values are evaluated when the function is declared, not when it is called.

It is stuck with the value it got the first time.

It does not "refresh" each time the function is called.

```
number = 5

def print_number(number: int = number) -> None:
    print(number)

number = 6 # This line does nothing
# Default value for the argument is stuck at 5

print_number(8) # 8
print_number() # 5
print(number)   # 6
```

Poll: What does this output? Why?

```
def send_message_and_cc_self(
    message: str, sender: str, recipients: list[str] = []) -> None:

    recipients.append(sender) # add sender to recipients

    for r in recipients:
        print(f"Sending '{message}' from {sender} to {r}")

send_message_and_cc_self("note to self", "Rasika")
send_message_and_cc_self("use RSA next time", "Eve", ["Alice", "Bob"])
send_message_and_cc_self("super secret", "admin")
```

This is an example of code that could look like it's doing one thing, when it's actually doing something else

Variable argument lists

Python allows us to have a function with an arbitrary number of arguments:

```
def print_args(*args: T) -> None:
    """Print each argument on a separate line"""
    for item in args:
        print(item)

print_args(1, 2, 3)
```

- `print_args()` can take any number of arguments
- They are of type `T`
- We can access them inside the function
 - each arg is an element in the tuple `args`
 - if there are no args, then `args` will be an empty tuple

Variable argument list, but with named arguments:

```
def print_args(**kwargs: T) -> None:
    """Print each argument on a separate line"""
    for argument_name, argument_value in kwargs.items():
        print(f'{argument_name}: {argument_value}')

print_args(a = 1, b = 2, c = 3)

a: 1
b: 2
c: 3
```

`**kwargs` stands for "keyword arguments", but you can name it anything you want.

We use two asterisks for `**kwargs` and one for `*args`.

Poll: How many arguments can I pass to this function?



User Guide API reference Building from source Development Release notes Learn More

Search Settings 2.3 (stable) GitHub

numpy.empty

numpy.empty_like

numpy.eye

numpy.identity

numpy.ones

numpy.ones_like

numpy.zeros

numpy.zeros_like

numpy.full

numpy.full_like

numpy.array

numpy.asarray

numpy.asanyarray

numpy.ascontiguousarray

NumPy reference

Routines and objects by topic

Array creation routines

numpy.fromfunction

numpy.fromfunction

numpy.fromfunction(*function*, *shape*, *, *dtype*=<class 'float'>, *Like*=None, ***kwargs*) [\[source\]](#)

Construct an array by executing a function over each coordinate.

The resulting array therefore has a value `fn(x, y, z)` at coordinate `(x, y, z)`.

Parameters:

function : *callable*

The function is called with N parameters, where N is the rank of `shape`. Each parameter represents the coordinates of the array varying along a specified axis. For example, if `shape` were `(2, 2)`, then the parameters would be `array([[0, 1], [0, 1]])`.

On this page

fromfunction

a. 0

b. 1

c. 2

d. 3

e. 4

f. 6

g. 10

The Single Responsibility Principle

A core principle for writing code is that every component of our code must have a single purpose. This makes our code easier to read, test, and maintain.

Component	Does not Follow Single Responsibility Principle
Variable	<code>age_or_nonexistent: int = -1 # negative if nonexistent</code>
Function	<code>def read_file_compute_average_print_score(filename: str) -> None:</code>
Class	<code>class FileManagerAndOutputFormatter</code>

Abstraction

We organize lines of code into functions, functions into classes, classes into...

"Abstraction": the procedure of grouping more granular things into less granular groups

Benefits of abstraction we already saw in functions:

- reuse code without redundancy
- hide implementation details
- break down a problem into smaller, more manageable pieces

The same applies to the idea of putting methods into classes.

Abstraction

Hard to debug or modify:

```
length: int = 5
width: int = 3

print(get_area_of_rectangle(
    length, width))

length = 6
width = 4

print(get_perimeter_of_rectangle(
    length, width))
```

Put relevant perimeter and area methods into a class for each shape:

```
table: Rectangle = Rectangle(5, 3)
print(table.area())

print(Rectangle(6, 4).perimeter())

chair: Square = Square(4)
print(chair.area())
```

Benefits of abstraction:

- We can use the same code multiple times without re-writing it
- It's easier to read
- It's easier to maintain and adapt the code later on
- It's easier to test behaviors in isolation
- Each variable, function, and class has a single, clear responsibility

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**